

[← Back to Blog](#)[AI](#) [Mastra](#) [A2A Protocol](#) [Telex](#) [Workflow Automation](#)

Building Intelligent Workflows: Integrating Mastra A2A Protocol with Telex

Phoenix • October 26, 2025 • 12 min read

Building Intelligent Workflows: Integrating Mastra A2A Protocol with Telex

In the rapidly evolving landscape of AI agents and workflow automation, interoperability between different AI systems has become crucial. This is where the **Agent-to-Agent (A2A) protocol** comes in, and today I'll walk you through how we integrated Mastra's A2A implementation with Telex, our organization's workflow automation platform.

What is Mastra A2A?

Mastra's Agent-to-Agent (A2A) protocol is a standardized communication layer that allows AI agents to interact seamlessly across different platforms. Built on the JSON-RPC 2.0 specification, it provides a robust framework for agent discovery, task execution, and message exchange.

The A2A protocol ensures that agents can:

- Communicate using a standardized message format
- Execute tasks with proper context management
- Maintain conversation history across interactions
- Return structured artifacts and results

The Architecture

Our implementation consists of three main components:

1. The Mastra Agent

First, we created a weather agent using Mastra's agent framework. This agent serves as our AI worker that processes requests and provides weather information.

typescript



```
import { Agent } from '@mastra/core/agent';
import { Memory } from '@mastra/memory';
import { LibSQLStore } from '@mastra/libsql';
import { weatherTool } from '../tools/weather-tool';

export const weatherAgent = new Agent({
  name: 'Weather Agent',
  instructions: `
    You are a helpful weather assistant that provides accurate weather information and can help planning

    Your primary function is to help users get weather details for specific locations. When responding:
    - Always ask for a location if none is provided
    - If the location name isn't in English, please translate it
    - Include relevant details like humidity, wind conditions, and precipitation
    - Keep responses concise but informative
    - If the user asks for activities and provides the weather forecast, suggest activities based on the

    Use the weatherTool to fetch current weather data.
  `,
  model: 'google/gemini-2.0-flash',
  tools: { weatherTool },
  memory: new Memory({
    storage: new LibSQLStore({
      url: 'file:../mastra.db',
    }),
  }),
});
```

2. The Weather Tool

The agent uses a custom tool to fetch real-time weather data from the Open-Meteo API:

typescript



```
import { createTool } from '@mastra/core/tools';
import { z } from 'zod';

export const weatherTool = createTool({
  id: 'get-weather',
  description: 'Get current weather for a location',
  inputSchema: z.object({
    location: z.string().describe('City name'),
  }),
  outputSchema: z.object({
```

```

    temperature: z.number(),
    feelsLike: z.number(),
    humidity: z.number(),
    windSpeed: z.number(),
    windGust: z.number(),
    conditions: z.string(),
    location: z.string(),
  }),
  execute: async ({ context }) => {
    const geocodingUrl = `https://geocoding-api.open-meteo.com/v1/search?name=${encodeURIComponent(context.location)}`;
    const geocodingResponse = await fetch(geocodingUrl);
    const geocodingData = await geocodingResponse.json();

    if (!geocodingData.results?.[0]) {
      throw new Error(`Location '${context.location}' not found`);
    }

    const { latitude, longitude, name } = geocodingData.results[0];
    const weatherUrl = `https://api.open-meteo.com/v1/forecast?latitude=${latitude}&longitude=${longitude}`;

    const response = await fetch(weatherUrl);
    const data = await response.json();

    return {
      temperature: data.current.temperature_2m,
      feelsLike: data.current.apparent_temperature,
      humidity: data.current.relative_humidity_2m,
      windSpeed: data.current.wind_speed_10m,
      windGust: data.current.wind_gusts_10m,
      conditions: getWeatherCondition(data.current.weather_code),
      location: name,
    };
  },
});

```

3. The A2A Route Handler

The crucial piece that bridges Mastra agents with the A2A protocol is our custom route handler. This handler wraps agent responses in proper A2A format with artifacts and history management:

typescript



```

import { registerApiRoute } from '@mastra/core/server';
import { randomUUID } from 'crypto';

export const a2aAgentRoute = registerApiRoute('/a2a/agent/:agentId', {
  method: 'POST',
  handler: async (c) => {
    try {
      const mastra = c.get('mastra');
      const agentId = c.req.param('agentId');

      // Parse JSON-RPC 2.0 request
      const body = await c.req.json();
    } catch (error) {
      // Handle error
    }
  }
});

```

```

const { jsonrpc, id: requestId, method, params } = body;

// Validate JSON-RPC 2.0 format
if (jsonrpc !== '2.0' || !requestId) {
  return c.json({
    jsonrpc: '2.0',
    id: requestId || null,
    error: {
      code: -32600,
      message: 'Invalid Request: jsonrpc must be "2.0" and id is required'
    }
  }, 400);
}

const agent = mastra.getAgent(agentId);
if (!agent) {
  return c.json({
    jsonrpc: '2.0',
    id: requestId,
    error: {
      code: -32602,
      message: `Agent '${agentId}' not found`
    }
  }, 404);
}

// Extract messages from params
const { message, messages, contextId, taskId, metadata } = params || {};

let messagesList = [];
if (message) {
  messagesList = [message];
} else if (messages && Array.isArray(messages)) {
  messagesList = messages;
}

// Convert A2A messages to Mastra format
const mastraMessages = messagesList.map((msg) => ({
  role: msg.role,
  content: msg.parts?.map((part) => {
    if (part.kind === 'text') return part.text;
    if (part.kind === 'data') return JSON.stringify(part.data);
    return '';
  }).join('\n') || ''
})));

// Execute agent
const response = await agent.generate(mastraMessages);
const agentText = response.text || '';

// Build artifacts array
const artifacts = [
  {
    artifactId: randomUUID(),
    name: `${agentId}Response`,
    parts: [{ kind: 'text', text: agentText }]
  }
];

// Add tool results as artifacts
if (response.toolResults && response.toolResults.length > 0) {

```

```

    artifacts.push({
      artifactId: randomUUID(),
      name: 'ToolResults',
      parts: response.toolResults.map((result) => ({
        kind: 'data',
        data: result
      })))
    });
  }

  // Build conversation history
  const history = [
    ...messagesList.map((msg) => ({
      kind: 'message',
      role: msg.role,
      parts: msg.parts,
      messageId: msg.messageId || randomUUID(),
      taskId: msg.taskId || taskId || randomUUID(),
    })),
    {
      kind: 'message',
      role: 'agent',
      parts: [{ kind: 'text', text: agentText }],
      messageId: randomUUID(),
      taskId: taskId || randomUUID(),
    }
  ];

  // Return A2A-compliant response
  return c.json({
    jsonrpc: '2.0',
    id: requestId,
    result: {
      id: taskId || randomUUID(),
      contextId: contextId || randomUUID(),
      status: {
        state: 'completed',
        timestamp: new Date().toISOString(),
        message: {
          messageId: randomUUID(),
          role: 'agent',
          parts: [{ kind: 'text', text: agentText }],
          kind: 'message'
        }
      },
      artifacts,
      history,
      kind: 'task'
    }
  });

} catch (error) {
  return c.json({
    jsonrpc: '2.0',
    id: null,
    error: {
      code: -32603,
      message: 'Internal error',
      data: { details: error.message }
    }
  }, 500);
}

```

```
}  
}  
});
```

4. Registering with Mastra

Finally, we register everything with the Mastra instance:

typescript



```
import { Mastra } from '@mastra/core/mastra';  
import { PinoLogger } from '@mastra/loggers';  
import { LibSQLStore } from '@mastra/libsql';  
import { weatherAgent } from './agents/weather-agent';  
import { a2aAgentRoute } from './routes/a2a-agent-route';  
  
export const mastra = new Mastra({  
  agents: { weatherAgent },  
  storage: new LibSQLStore({ url: ":memory:" }),  
  logger: new PinoLogger({  
    name: 'Mastra',  
    level: 'debug',  
  }),  
  observability: {  
    default: { enabled: true },  
  },  
  server: {  
    build: {  
      openAPIDocs: true,  
      swaggerUI: true,  
    },  
    apiRoutes: [a2aAgentRoute]  
  }  
});
```

Deployment to Mastra Cloud

Once your Mastra agent is ready, deploying to Mastra Cloud is straightforward:

bash



```
# Build the project  
pnpm run build  
  
# Deploy to Mastra Cloud  
mastra deploy
```

After deployment, your agent gets a unique A2A endpoint URL:

`https://telex-mastra.mastra.cloud/a2a/agent/weatherAgent`

Integration with Telex

Now comes the exciting part - integrating your Mastra agent with Telex workflows!

Step 1: Create an AI Co-Worker in Telex

In your Telex dashboard, navigate to the AI Co-Workers section and create a new co-worker.

Step 2: Define Your Workflow

In the workflow editor, paste the following workflow definition. The key component here is the **node definition** which tells Telex how to communicate with your Mastra agent:

json

```
{
  "active": false,
  "category": "Utilities",
  "description": "A workflow that gives weather information",
  "id": "s6C3u7y4vBaZww0G",
  "name": "phoenix_agent",
  "long_description": "
    You are a helpful weather assistant that provides accurate weather information and can help planning

    Your primary function is to help users get weather details for specific locations. When responding:
    - Always ask for a location if none is provided
    - If the location name isn't in English, please translate it
    - Include relevant details like humidity, wind conditions, and precipitation
    - Keep responses concise but informative
    - If the user asks for activities and provides the weather forecast, suggest activities based on the

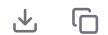
    Use the weatherTool to fetch current weather data.
  ",
  "short_description": "Get weather information for any location",
  "nodes": [
    {
      "id": "weather_agent",
      "name": "weather agent",
      "parameters": {},
      "position": [816, -112],
      "type": "a2a/mastra-a2a-node",
      "typeVersion": 1,
    }
  ]
}
```

```
    "url": "https://telex-mastra.mastra.cloud/a2a/agent/weatherAgent"
  }
],
"pinData": {},
"settings": {
  "executionOrder": "v1"
}
}
```

Understanding the Node Definition

The magic happens in the ``nodes`` array. Let's break down the critical parts:

json



```
{
  "id": "weather_agent",           // Unique identifier for this node
  "name": "weather agent",        // Display name
  "parameters": {},               // Node parameters (empty for basic A2A)
  "position": [816, -112],        // Visual position in workflow editor
  "type": "a2a/mastra-a2a-node",  // Node type (A2A integration)
  "typeVersion": 1,               // Version of the node type
  "url": "https://telex-mastra.mastra.cloud/a2a/agent/weatherAgent" // Your A2A endpoint
}
```

The most important field is the ``url`` - this is the A2A endpoint URL of your deployed Mastra agent. Everything else in the workflow JSON is primarily descriptive metadata about your agent's purpose and behavior.

How It Works

When a user interacts with your Telex workflow:

1. **User sends a message** through Telex (e.g., "What's the weather in Lagos?")
2. **Telex constructs an A2A request:**

json



```
{
  "jsonrpc": "2.0",
  "id": "request-001",
```



```
"method": "message/send",
"params": {
  "message": {
    "kind": "message",
    "role": "user",
    "parts": [
      {
        "kind": "text",
        "text": "What's the weather in Lagos?"
      }
    ],
    "messageId": "msg-001",
    "taskId": "task-001"
  },
  "configuration": {
    "blocking": true
  }
}
```

3. Your Mastra agent processes the request:

- Receives the A2A message
- Extracts the user's question
- Uses the weather tool to fetch data
- Generates a response

4. Returns an A2A-compliant response:

json



```
{
  "jsonrpc": "2.0",
  "id": "request-001",
  "result": {
    "id": "task-001",
    "contextId": "context-uuid",
    "status": {
      "state": "completed",
      "timestamp": "2025-10-26T10:30:00.000Z",
      "message": {
        "messageId": "response-uuid",
        "role": "agent",
        "parts": [
          {
            "kind": "text",
            "text": "The current weather in Lagos is 29°C with thunderstorms. Humidity is at 85%, with win
```

```

    }
  ],
  "kind": "message"
}
},
"artifacts": [
  {
    "artifactId": "artifact-uuid",
    "name": "weatherAgentResponse",
    "parts": [
      {
        "kind": "text",
        "text": "The current weather in Lagos is 29°C..."
      }
    ]
  },
  {
    "artifactId": "tool-uuid",
    "name": "ToolResults",
    "parts": [
      {
        "kind": "data",
        "data": {
          "temperature": 29.1,
          "feelsLike": 32.5,
          "humidity": 85,
          "windSpeed": 15,
          "conditions": "Thunderstorm",
          "location": "Lagos"
        }
      }
    ]
  }
],
"history": [...]
```

5. Telex displays the response to the user in a friendly format

Key Benefits

1. Standardization

The A2A protocol ensures that agents can communicate regardless of their underlying implementation. Your Mastra agent can seamlessly integrate with any A2A-compliant platform.

2. Proper Context Management

The protocol maintains conversation context, task IDs, and message history, enabling multi-turn conversations and stateful interactions.

3. Structured Artifacts

Responses include both human-readable text and structured data artifacts, making it easy to build complex workflows that chain multiple agents together.

4. Error Handling

JSON-RPC 2.0 compliance ensures standardized error responses, making debugging and monitoring straightforward.

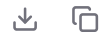
5. Flexibility

The same Mastra agent can be used across different platforms (Telex, custom applications, other A2A clients) without modification.

Testing Your Integration

You can test your A2A endpoint directly using curl:

bash



```
curl -X POST https://telex-mastra.mastra.cloud/a2a/agent/weatherAgent \
-H "Content-Type: application/json" \
-d '{
  "jsonrpc": "2.0",
  "id": "test-001",
  "method": "message/send",
  "params": {
    "message": {
      "kind": "message",
      "role": "user",
      "parts": [
        {
          "kind": "text",
          "text": "What is the weather in New York?"
        }
      ],
      "messageId": "msg-001",
      "taskId": "task-001"
    },
    "configuration": {
      "blocking": true
    }
  }
}
```

Conclusion

Integrating Mastra's A2A protocol with Telex demonstrates the power of standardized agent communication. By following this approach, you can:

- Build sophisticated AI agents with Mastra's framework
- Deploy them to the cloud with a single command
- Integrate them into complex workflows using Telex
- Maintain clean separation between agent logic and workflow orchestration

The A2A protocol is more than just a technical specification - it's a foundation for building the next generation of interconnected AI systems. Whether you're building weather assistants, data processors, or complex multi-agent systems, this pattern provides a robust and scalable solution.

Next Steps

Want to build your own A2A-enabled agents? Here are some ideas:

- **Customer Support Agent:** Handle FAQs and ticket routing
- **Data Analysis Agent:** Process and visualize datasets
- **Code Review Agent:** Analyze pull requests and suggest improvements
- **Content Generator:** Create marketing copy or documentation
- **Translation Agent:** Multi-language content translation

The possibilities are endless. Happy building!

Resources:

- [Mastra Documentation](#)
- [A2A Protocol Specification](#)
- [Telex Platform](#)
- [GitHub Repository - Mastra x Telex Example](#)

This implementation was built using Mastra v0.22.2 and deployed to Mastra Cloud in October 2025.

← Back to all posts

Share on X

PHOENIX